

Tasnet

Bài báo:

[tasnet.pdf](#)

Code github:

<https://github.com/kaituoxu/TasNet>

Các thành phần chính:

Encoder

Input:

mixture: [B,K,L]: tín hiệu âm thanh hỗn hợp

- B: Kích thước batch.
- K: Số đoạn (segment) trong mỗi mẫu.
- L: Số mẫu (samples) trong mỗi đoạn tín hiệu.

Output:

mixture_w: [B, K, N]: Biểu diễn đặc trưng của tín hiệu sau khi mã hóa.

norm_coef: [B, K, 1]: Hệ số chuẩn hóa, dùng trong bước giải mã để tái tạo tín hiệu gốc.

Chi tiết:

Khởi tạo (init):

```
# hyper-parameter
self.L = L
self.N = N
# Components
# Maybe we can impl 1-D conv by nn.Linear()?
self.conv1d_U = nn.Conv1d(L, N, kernel_size=1, stride=1, bias
self.conv1d_V = nn.Conv1d(L, N, kernel_size=1, stride=1, bias
```

- N là số kênh đầu ra (số lượng bộ lọc)

L2 Normalization:

Mục đích: Chuẩn hóa tín hiệu đầu vào để giảm độ biến thiên và tăng tính ổn định khi tính toán.

```
norm_coef = torch.norm(mixture, p=2, dim=2, keepdim=True)
norm_mixture = mixture / (norm_coef + EPS) # B x K x L
```

- L2 thay thế L (số mẫu) bằng giá trị chuẩn L2
- Chuẩn hóa tín hiệu: để giảm độ biến thiên và tăng tính ổn định
 - EPS là một giá trị nhỏ đưa vào để tránh trường hợp norm_coef bằng 0

Gate conv:

- Reshape: chuyển đổi tín hiệu norm_mixture (B,K,L) ghép B và K lại thành 1 chiều

```
norm_mixture = torch.unsqueeze(norm_mixture.view(-1, L), 2)
```

- Tích chập `conv1d_U` sử dụng Relu để loại bỏ giá trị âm

```
conv = F.relu(self.conv1d_U(norm_mixture)) # B*K x N x 1
```

- Tích chập `conv1d_V` sử dụng Sigmoid để chuẩn hóa đầu ra vào khoảng [0-1]

```
gate = torch.sigmoid(self.conv1d_V(norm_mixture)) # B*K x
```

- Tích từng phần tử giữa conv conv và gate gate để giữ lại những thông tin quan trọng

```
mixture_w = conv * gate # B*K x N x 1
```

- Điều chỉnh đầu ra

```
mixture_w = mixture_w.view(B, K, self.N) # B x K x N
```

Ý nghĩa của cơ chế 1D Gated Convolution

1. `conv1d_U` (Feature Extraction):

- Học các đặc trưng quan trọng từ tín hiệu đã chuẩn hóa.
- Đảm bảo đầu ra không âm bằng hàm kích hoạt **ReLU**.

2. `conv1d_V` (Gating Mechanism):

- Học cổng điều chỉnh để quyết định phần thông tin nào từ `conv1d_U` được truyền qua.

`conv1d_U` \text{conv1d_U}

- Giới hạn giá trị trong khoảng $[0,1]$, giúp tăng tính ổn định và khả năng học.

$[0,1]$ $[0, 1]$

3. Phép nhân (Element-wise Multiplication):

- Cơ chế gating giúp lọc bỏ nhiễu và thông tin không cần thiết, chỉ giữ lại các đặc trưng quan trọng.

Separator

• Vai trò của Separator

- Dựa trên biểu diễn tín hiệu được mã hóa (`mixture_w`) từ Encoder, tách riêng các nguồn tín hiệu thông qua việc ước lượng các **mặt nạ** (`est_mask`).

- Mặt nạ này được sử dụng để phân bổ các phần tương ứng của tín hiệu hỗn hợp (mixture_w) cho từng nguồn.
- Layer Normalization
 - Chuẩn hóa tín hiệu mixture_w để ổn định việc huấn luyện, giúp giảm độ biến thiên khi học các đặc trưng, tăng tốc độ huấn luyện
- LSTM: LSTM học các phụ thuộc dài hạn trong tín hiệu dọc theo trục thời gian K .
 - Là một loại mạng RNN
 - Giải quyết các vấn đề về **gradient vanishing** (mất gradient) và **gradient exploding** (bùng nổ gradient)
 - **Ghi nhớ thông tin quan trọng** trong thời gian dài và **bỏ qua thông tin không quan trọng**.
 - Mỗi đơn vị có 3 trạng thái chính:
 - **ô nhớ (Cell State)**: lưu trữ các thông tin lâu dài
 - **ẩn (Hidden State)**: thông tin ngắn hạn tại thời điểm hiện tại
 - **cổng (Gate)**: điều khiển việc lưu trữ hay quên thông tin
 - **Forget Gate (Cổng quên)**: quyết định thông tin nào trong ô nhớ không cần thiết
 - **Input Gate (Cổng ghi nhớ mới)**: Quyết định phần nào của thông tin mới sẽ được thêm vào ô nhớ.
 - **Output Gate (Cổng đầu ra)**: Quyết định sử dụng phần nào của trạng thái ô nhớ cho đầu ra hiện tại.
- Fully Connected: Tạo est_mask từ LSTM
- Softmax: Chuẩn hóa est_mask

Decoder

Mục đích

Bước Decoder trong TasNet có nhiệm vụ tái tạo lại tín hiệu âm thanh gốc của từng nguồn từ biểu diễn đã mã hóa (mixture_w) và các mặt nạ ước lượng (est_mask).

Các bước hoạt động

1. Áp dụng mặt nạ lên biểu diễn mã hóa:

- Kết hợp `mixture_w` với `est_mask` để tách riêng các trọng số mã hóa (`source_w`) của từng nguồn:

-

$$\text{source_w}_i = \text{mixture_w} \cdot \text{est_mask}_i$$

- $\text{mixture_w} \in \mathbb{R}^{B \times K \times N}$: Biểu diễn mã hóa đầu vào.
- $\text{est_mask}_i \in \mathbb{R}^{B \times K \times N}$: Mặt nạ của nguồn i .
- $\text{source_w}_i \in \mathbb{R}^{B \times K \times N}$: Biểu diễn mã hóa của nguồn i .

```
source_w = torch.unsqueeze(mixture_w, 2) * est_mask
```

2. Giải mã tín hiệu:

- Sử dụng một lớp tích chập ngược (transposed convolution) để chuyển đổi `source_w` từ biểu diễn mã hóa về dạng sóng âm thanh gốc:

$$\text{source}_i = \text{source_w}_i \cdot \text{basis_signals}$$

- $\text{basis_signals} \in \mathbb{R}^{N \times L}$: Bộ lọc cơ sở được học từ dữ liệu.
- $\text{source}_i \in \mathbb{R}^{B \times K \times L}$: Dạng sóng âm thanh của nguồn i .

```
est_source = self.basis_signals(source_w)
```

3. Phục hồi biên độ gốc (Reverse Normalization):

- Nhân tín hiệu tái tạo với hệ số chuẩn hóa (norm_coef) được tính trong bước Encoder để khôi phục biên độ ban đầu:

$$\text{source}_i = \text{source}_i \cdot \text{norm_coef}$$

```
est_source = est_source * norm_coef
```

4. Ghép các đoạn tín hiệu:

- Tập hợp lại tất cả các đoạn tín hiệu K để tạo ra tín hiệu hoàn chỉnh của mỗi nguồn.

```
est_source = est_source.permute((0, 2, 1, 3)).contiguous()
```